Rešitve primerov

Rešitve primerov iz mape `Primeri`

Opomba: slike so fotografije, zato so nekateri robovi nalog slabše berljivi. Kjer je potrebno, sem zapisal predpostavko.

`image0.jpg` - 2. izpitni rok 2024, 1. naloga

Naloga

Podani so trije algoritmi:

- A: urejanje z zlivanjem.
- B: najcenejša pot v usmerjenem polnem grafu z n točkami s topološkim urejanjem.
- C: osnovno množenje matrik.

a) Asimptotske zahtevnosti

- `T_A = Θ(n log n)` - merge sort razdeli tabelo na polovice in na vsakem nivoju opravi `Θ(n)` zlivanja.
- `T_B = Θ(n^2)` - poln graf ima `m=Θ(n^2)` povezav, topološko urejanje in sproščanje povezav porabita `Θ(n+m)=Θ(n^2)`.
- `T_C = Θ(n^3)` - osnovno množenje dveh `n x n` matrik ima tri gnezdene zanke.

b) Kolikšen `n` rešimo v 1 sekundi

Predpostavka: 1 GHz pomeni približno `10^9` osnovnih operacij/s, konstanta `C=2`, zato mora veljati `f(n) <= 5*10^8`.

- `n log2 n <= 5*10^8` -> `n ≈ 20.580.553`.
- `n^2 <= 5*10^8` -> `n ≈ 22.360`.
- `n^3 <= 5*10^8` -> `n ≈ 793`.

Pazi: to je groba teoretična ocena, ne realna meritev.

`IMG_5396.jpg` - 1. izpitni rok 2024, 1. naloga

Naloga

Oceniti čas urejanja imen vseh prebivalcev Kitajske, približno `n = 1.412.000.000`, za:

- kvadratični algoritem,
- algoritem `O(n log n)`.

Predpostavka: 1 osnovna operacija traja približno 1 ns, primerjava imen ima konstantno ceno.

Kvadratično urejanje

`n^2 = (1.412*10^9)^2 ≈ 1.993*10^18` operacij.

Pri 10^9 operacij/s:

1.993*10^18 / 10^9 = 1.993*10^9 s.

To je približno 63 let.

n log n urejanje

log2(1.412*10^9) ≈ 30.4.

n log2 n ≈ 1.412*10^9 * 30.4 ≈ 4.29*10^10 operacij.

Pri 10^9 operacij/s je to približno 43 sekund.

Zaključek: razlika med n^2 in n log n pri milijardnih vhodih ni “malo počasneje”, ampak razlika med minuto in življenjem.

PXL_20230619_081001816.jpg - stopnice

A) Samo koraki dolžine 1

Za n stopnic obstaja samo en način:

1+1+...+1.

Odgovor: 1.

B) 4 stopnice, koraki 1 ali 2

Možnosti:

- 1+1+1+1
- 1+1+2
- 1+2+1
- 2+1+1
- 2+2

Odgovor: 5.

C) 5 stopnic, koraki 1 ali 2

Rekurenca:

f(n)=f(n-1)+f(n-2)

f(0)=1, f(1)=1, f(2)=2, f(3)=3, f(4)=5, f(5)=8.

Odgovor: 8.

D) Splošna formula

f(n)=f(n-1)+f(n-2), z osnovama f(0)=1, f(1)=1.

Razlaga: zadnji korak je ali dolžine 1, zato prej stojimo na `n-1`, ali dolžine 2, zato prej stojimo na `n-2`.

E) Učinkovit program

```
stStopnic(n):
    if n == 0 or n == 1:
        return 1
    a = 1
    b = 1
    for i = 2..n:
        c = a + b
        a = b
        b = c
    return b
```

Čas $\Theta(n)$, prostor $\Theta(1)$.

`20240620_100553.jpg` - tabela tabel

a) Koliko tabel potrebujemo za 2024 elementov

`2024 = 1024 + 512 + 256 + 128 + 64 + 32 + 8`

Binarno so zasedene tabele:

`A_10, A_9, A_8, A_7, A_6, A_5, A_3`

Odgovor: `7` zasedenih tabel.

Opomba: če bi šteli vse nivoje od `A_0` do `A_10`, bi bilo nivojev `11`, vendar je po strukturi “tabela tabel” običajno mišljeno število zasedenih tabel.

b) Koliko elementov lahko dodamo, da se število uporabljenih tabel ne spremeni

Če “uporabljene tabele” pomeni zasedene tabele, potem že pri dodajanju enega elementa:

`2024 + 1 = 2025`

se vklopi še bit `A_0`, zato se število zasedenih tabel spremeni. Odgovor: `0`.

Če bi naloga mislila število nivojev do največje tabele (`A_0` do `A_10`), lahko dodamo do `2047-2024 = 23` elementov, preden potrebujemo `A_11`.

c) Časovna zahtevnost iskanja

Zasedenih je `O(log n)` urejenih tabel. V vsaki iščemo z bisekcijo.

Skupaj:

`O(log n + log n + ... ) = O(log^2 n)`.

d) Vstavljanje v strukturo

Začetek:

- `A_0 = [9]`

- `A_1 = [7, 8]`

Dodamo `1, 5, 6, 2, 4, 3, 0, 10`.

Postopek je kot binarni števec: če je trenutna tabela zasedena, jo zlijemo s prenosom naprej.

Končno stanje:

- `A_0 = [10]`
- `A_1 = [0, 3]`
- `A_2 = []`
- `A_3 = [1, 2, 4, 5, 6, 7, 8, 9]`

Preverjanje: skupaj `1 + 2 + 8 = 11` elementov, kar ustreza začetnim 3 + dodanim 8.

`PXL_20230619_081016189.jpg` - minimalno vpeto drevo

A) Matrika cen

Vozlišča so `1..7`. Neobstoječo povezavo označim z ∞, diagonalo z `0`.

	1	2	3	4	5	6	7
1	0	15	68	∞	∞	84	49
2	15	0	99	30	∞	∞	∞
3	68	99	0	93	72	∞	∞
4	∞	30	93	0	79	∞	38
5	∞	∞	72	79	0	88	∞
6	84	∞	∞	∞	88	0	90
7	49	∞	∞	38	∞	90	0

B) Primov algoritem iz vozlišča 1

Koraki:

1. Start `1`.
2. Najcenejša povezava iz `1`: `1-2 (15)`.
3. Iz množice `{1,2}` dodamo `2-4 (30)`.
4. Iz `{1,2,4}` dodamo `4-7 (38)`.
5. Iz `{1,2,4,7}` dodamo `1-3 (68)`.
6. Iz `{1,2,3,4,7}` dodamo `3-5 (72)`.
7. Iz `{1,2,3,4,5,7}` dodamo `1-6 (84)`.

MST povezave:

`{1-2, 2-4, 4-7, 1-3, 3-5, 1-6}`

Skupna cena:

`15 + 30 + 38 + 68 + 72 + 84 = 307`.

Vrstni red obiskovanja: `1, 2, 4, 7, 3, 5, 6`.

C) Ali je rešitev enolična

Da. Vse uteži na povezavah so različne, zato je minimalno vpeto drevo enolično.

D) Učinkovita implementacija Prima

Graf naj bo predstavljen s seznamom sosedov, množica kandidatov pa s prioritetno vrsto oziroma min-kopico. Tako hitro dobimo najcenejšo naslednjo povezavo.

`IMG_5395.jpg` - preskočni seznam in Bellman-Ford

2. naloga - preskočni seznam

a) Uporaba generatorja naključnih števil

Pri vstavljanju elementa določimo njegovo višino z meti kovanca:

- element je vedno na nivoju 0,
- dokler generator vrača `C`, element promoviramo za en nivo,
- ko vrne `M`, se ustavimo.

Predpostavka: `C = uspeh/promocija`, `M = stop`.

b) Trditve

Pričakovano iskanje in vstavljanje v preskočni seznam sta $\Theta(\log n)$.

- $i(n) \in \Omega(n^2)$ - NE. $\log n$ ne raste vsaj tako hitro kot n^2.
- $v(n) \in O(n)$ - DA. Ker $\log n \in O(n)$.

c) Vstavljanje `7, 10, 9, 3, 5, 8, 6`

Predpostavka za zaporedje metov: `C C M M C M M M C C M C M ...`

Višine:

- `7`: `C,C,M` -> nivoji 0,1,2
- `10`: `M` -> nivo 0
- `9`: `C,M` -> nivoji 0,1
- `3`: `M` -> nivo 0
- `5`: `M` -> nivo 0
- `8`: `C,C,M` -> nivoji 0,1,2
- `6`: `C,M` -> nivoji 0,1

Končni preskočni seznam:

```
nivo 2: 7 -------- 8
nivo 1: 6 -- 7 -- 8 -- 9
nivo 0: 3 -- 5 -- 6 -- 7 -- 8 -- 9 -- 10
```

5. naloga - Bellman-Ford

Prebrane povezave:

- `1->2 (3)`, `1->3 (1)`, `1->6 (4)`
- `2->4 (1)`, `2->6 (7)`
- `3->5 (1)`, `3->6 (3)`
- `4->5 (3)`, `4->6 (5)`
- `5->6 (1)`

Iščemo razdalje od vozlišč `1..6` do vozlišča `6`.

Najkrajše poti:

- `d_6 = 0`
- `d_5 = 1` po `5->6`
- `d_4 = 4` po `4->5->6`
- `d_3 = 2` po `3->5->6`
- `d_2 = 5` po `2->4->5->6`
- `d_1 = 3` po `1->3->5->6`

Vektor:

`d = [3, 5, 2, 4, 1, 0]`

PXL_20230619_080959115.jpg - preskočni seznam

b) Trditve

Pričakovano iskanje in vstavljanje sta $\Theta(\log n)$.

- $i(n) \in \Omega(n \log n)$ - NE.
- $v(n) \in O(n)$ - DA.

c) Vstavljanje `9, 7, 4, 1, 3, 5, 2, 8`

Predpostavka: `C` promovira, `M` ustavi. Prebrano zaporedje: `M C C C M M C M M M C C M C M ...`.

Višine:

- 9: `M` -> nivo 0
- 7: `C,C,C,M` -> nivoji 0,1,2,3
- 4: `M` -> nivo 0
- 1: `C,M` -> nivoji 0,1
- 3: `M` -> nivo 0
- 5: `M` -> nivo 0
- 2: `C,C,M` -> nivoji 0,1,2
- 8: `C,M` -> nivoji 0,1

Končni seznam:

```
nivo 3: 7
nivo 2: 2 -------- 7
nivo 1: 1 -- 2 -- 7 -- 8
nivo 0: 1 -- 2 -- 3 -- 4 -- 5 -- 7 -- 8 -- 9
```

IMG_5392.jpg - verižno množenje matrik

1) Trditve

a) "Skupno število operacij ni odvisno od postavitve oklepajev."

Odgovor: NE. Množenje matrik je asociativno glede rezultata, ne glede cene. Različni oklepaji lahko pomenijo zelo različna števila skalarnih množenj.

b) "Velikost matrike X=A_1...A_k ni odvisna od postavitve oklepajev."

Odgovor: DA, če je produkt veljaven. Končna dimenzija je določena s prvo vrstico prve matrike in zadnjim stolpcem zadnje matrike.

2) Matrike

Dimenzije:

- A = 4x6
- B = 3x8
- C = 6x3
- D = 8x2

a) Kateri vrstni red je pravilen

Možnosti:

- ABCD: ne, ker A(4x6) in B(3x8) nista združljivi.
- BDAC: ne, ker po BD dobimo 3x2, tega ne moremo množiti z A(4x6).
- ACBD: da. A*C = 4x3, potem *B = 4x8, potem *D = 4x2.
- CBAD: ne, ker po CB dobimo 6x8, tega ne moremo množiti z A(4x6).

Pravilno: ACBD.

b) Najmanjše število produktov

Za verigo A C B D so dimenzije:

```
4, 6, 3, 8, 2
```

Najboljša postavitev oklepajev:

```
A * (C * (B * D))
```

Cena:

- `B*D`: `3*8*2 = 48`, rezultat `3x2`
- `C*(BD)`: `6*3*2 = 36`, rezultat `6x2`
- `A*(C(BD))`: `4*6*2 = 48`, rezultat `4x2`

Skupaj:

`48 + 36 + 48 = 132`.

`20240620_100533.jpg` - nahrbtnik

Podatki:

predmet	1	2	3	4	5
velikost	2	5	3	8	1
vrednost	3	5	5	9	3

Kapaciteta: `V=8`.

a) Dovoljeno rezanje predmetov

Razmerja `v/w`:

- predmet 1: `3/2 = 1.5`
- predmet 2: `5/5 = 1`
- predmet 3: `5/3 ≈ 1.67`
- predmet 4: `9/8 = 1.125`
- predmet 5: `3/1 = 3`

Vrstni red: `5, 3, 1, 4, 2`.

Vzamemo:

- predmet 5 v celoti: velikost 1, vrednost 3
- predmet 3 v celoti: velikost 3, vrednost 5
- predmet 1 v celoti: velikost 2, vrednost 3
- ostane kapaciteta 2, vzamemo `2/8` predmeta 4: vrednost `9*(2/8)=2.25`

Skupna vrednost:

`3 + 5 + 3 + 2.25 = 13.25`.

b) 0/1 nahrbtnik

Najboljša vrednost je `11`.

Optimalni izbiri:

- predmeti `{1, 3, 5}`: velikost 2+3+1=6, vrednost 3+5+3=11
- ali `{1, 2, 5}`: velikost 2+5+1=8, vrednost 3+5+3=11

Preverjanje: predmet 4 sam da vrednost 9, kombinacija 2+3 ima vrednost 10, zato 11 ostane najboljše.

`20240620_100530.jpg` - zgoščena tabela

Zaporedje: `ALGORITMI`.

`h(k) = (k*p) mod m`, `p=11`.

a) Veriženje, elementi v istem košu kot BST

Za `m=5` velja `11 mod 5 = 1`, zato je `h(k)=k mod 5`.

ASCII ostanki:

- `A=65 -> 0`
- `L=76 -> 1`
- `G=71 -> 1`
- `O=79 -> 4`
- `R=82 -> 2`
- `I=73 -> 3`
- `T=84 -> 4`
- `M=77 -> 2`
- `I=73 -> 3`

Končno:

- koš 0: `A`
- koš 1: BST z `L` koren, `G` levo
- koš 2: BST z `R` koren, `M` levo
- koš 3: `I` oziroma pri dovoljenih duplikatih `I` z drugim `I`
- koš 4: BST z `O` koren, `T` desno

Opomba: če struktura predstavlja slovar, drugi `I` posodobi obstoječ ključ. Če naloga obravnava črke kot zaporedne vnose, se drugi `I` shrani kot duplikat po dogovorjenem pravilu.

b) Linearno prenaslavljanje, ob polni tabeli podvojimo velikost

Najprej `m=5`:

- `A -> 0`
- `L -> 1`
- `G -> 1`, zasedeno, gre na 2
- `O -> 4`
- `R -> 2`, zasedeno, gre na 3

Tabela je polna. Pred vstavljanjem `I` povečamo na `m=10` in rehashamo:

- `A=65 -> 5`
- `L=76 -> 6`
- `G=71 -> 1`
- `O=79 -> 9`

- `R=82 -> 2`

Nadaljujemo:

- `I=73 -> 3`
- `T=84 -> 4`
- `M=77 -> 7`
- drugi `I=73`: če dovolimo duplikat, linearno išče `3,4,5,6,7,8` in gre na `8`; če gre za slovar, posodobi obstoječ `I` na indeksu 3.

Končna tabela pri dovoljenih duplikatih:

indeks	0	1	2	3	4	5	6	7	8	9
vrednost	-	G	R	I	T	A	L	M	I	O

`20240620_100452.jpg` - višinska mapa

Naloga išče najcenejšo pot od celice 0 do 24 po 5x5 mreži z dovoljenimi premiki naravnost in diagonalno.

Prebrana višinska tabela:

```
0:7   1:0   2:3   3:2   4:5
5:3   6:3   7:4   8:1   9:1
10:2  11:7  12:5  13:5  14:6
15:3  16:0  17:5  18:5  19:5
20:7  21:5  22:1  23:6  24:4
```

Pomembna nejasnost: fotografija pravi, da je pri spustu cena negativna, kar lahko ustvari negativne cikle. Če so cikli dovoljeni, najcenejša pot ni dobro definirana, ker lahko s ponavljanjem cikla ceno poljubno zmanjšamo.

Zato sta smiselni dve interpretaciji:

Interpretacija 1 - pot brez ponavljanja vozlišč

Uporabimo Bellman-Ford samo kot idejo sproščanja, vendar moramo prepovedati ponavljanje vozlišč. Zaradi negativnih spustov je treba v rešitvi izrecno paziti na cikle. Natančnega enoličnega rezultata iz slike ne morem varno potrditi brez jasnejšega pravila o ciklih.

Interpretacija 2 - spust ima pozitivno ceno absolutne višinske razlike

Če spust štejemo kot pozitivni napor, dobimo najcenejšo pot:

`0 -> 6 -> 7 -> 13 -> 18 -> 24`

Cena:

- `0->6`: diagonala, `|3-7|=4`, cena `0.7*4 = 2.8`
- `6->7`: vzpon za 1, cena `1^2 = 1`
- `7->13`: diagonala, vzpon za 1, cena `0.7*1^2 = 0.7`
- `13->18`: enaka višina, cena `0`
- `18->24`: diagonala, spust za 1, cena `0.7*1 = 0.7`

Skupaj: `5.2`.

`20240610_100725.jpg` - rdeče-črna drevesa

a) DA/NE

A) "Rdeče-črno drevo je dvojiško drevo."

DA. Rdeče-črno drevo je dvojiško iskalno drevo z dodatnimi barvnimi pravili.

B) "Če RČ drevo vsebuje n elementov, bo iskanje potrebovalo `O(n^2)` časa."

DA kot zelo ohlapna zgornja meja, vendar nenatančno. Pravilna tesna meja je `O(log n)`, ker je višina RČ drevesa `O(log n)`.

C) "Če RČ in AVL drevo vsebujeta iste elemente, bo globina RČ drevesa večja od globine AVL drevesa."

NE. AVL je strožje uravnotežen, zato ima pogosto manjšo višino, ni pa nujno za vsako konkretno množico in obliko drevesa, da je RČ strogo globlje.

D) "Najmanjši element v AVL drevesu je vedno v korenu."

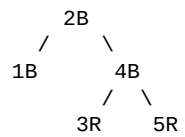
NE. V BST/AVL je najmanjši element skrajno levo.

E) "Število rdečih elementov v RČ drevesu je za ena manjše od števila črnih."

NE. Pravila določajo črno višino in prepoved dveh zaporednih rdečih, ne fiksnega razmerja rdečih in črnih vozlišč.

b) Vstavimo 9, 8, 7 v dano RČ drevo

Začetek:



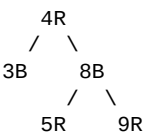
Vstavi 9:

- 9 gre desno od 5.
- Nov element je rdeč.
- Starš 5 je rdeč, stric 3 je rdeč -> prebarvanje: 3 in 5 postaneta črna, 4 postane rdeč.
- Koren 2 ostane črn.

Vstavi 8:

- 8 gre levo od 9.
- Starš 9 je rdeč, stric je črn/prazen.
- Primer RL glede na 5: najprej desna rotacija pri 9, nato leva rotacija pri 5, zamenjava barv.

Desno poddrevo pri 4 postane približno:



Vstavi 7:

- 7 gre desno od 5.
- Starš 5 je rdeč, stric 9 je rdeč -> prebarvanje: 5 in 9 črna, 8 rdeča.
- Ker je 8 otrok rdečega 4, popravljamo še pri 4/8 glede na koren 2.

Končna oblika je odvisna od uporabljene konvencije NIL-listov in zaporedja popravljanja, ključni postopki pa so: prebarvanje pri rdečem stricu, nato rotacija/prebarvanje pri dveh zaporednih rdečih vozliščih.

`PXL_20230619_081022358.jpg` - drevo

Fotografija je delno odrezana, zato je del B rekonstruiran po vidnem drevesu.

A) Trditve

Polno dvojiško iskalno drevo `T` globine `h`; odstranimo vsak drug list z leve proti desni in dobimo `T'`.

a) "Vsako polno dvojiško drevo je AVL drevo."

DA, če "polno" pomeni popolno drevo, kjer so vsi nivoji zapolnjeni. Višini levega in desnega poddrevesa sta povsod enaki.

b) Če je `t(n)` število listov drevesa `T`, potem `t(n) ∈ O(n^2)`.

DA. Velja celo `t(n) <= n`, zato je tudi `O(n^2)`.

c) "`T'` je AVL drevo."

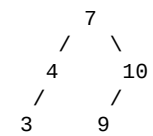
DA po interpretaciji, da odstranimo izmenično liste tako, da vsak starš na zadnjem nivoju izgubi natanko enega otroka. Višinska razlika ostane največ 1.

d) "Število listov drevesa `T'` je `2^(h-1)-1`."

NE. Če ima popolno drevo globine `h` `2^h` listov, po odstranitvi vsakega drugega ostane `2^(h-1)` listov.

B) Vstavljanje 2 in 8 v AVL

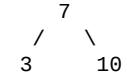
Vidno začetno drevo:

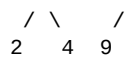


Vstavimo 2:

- 2 gre levo od 3.
- Vozlišče 4 postane neuravnoteženo tipa LL.
- Naredimo desno rotacijo pri 4.

Po vstavljanju 2:





Vstavimo 8:

- 8 gre levo od 9.
- Vozlišče 10 postane neuravnoteženo tipa LL.
- Naredimo desno rotacijo pri 10.

Končno:

